



Instituto Politécnico Nacional

Escuela Superior de Computo

Sistemas en Chip

Maestro: Perez Perez Jose Juan

Practica 1:

Comparador/ASCII-HEXADECIMAL

Integrantes:

- Ayala Fuentes Sunem Gizeht
- Guevara Badillo Areli Alejandra
- León Pérez May Carolina

Equipo: 1

Grupo 6CV1

Introducción

“Programación del ATmega8535 en ensamblador”

El ATmega8535 es un microcontrolador de la familia AVR desarrollado por Atmel. Pertenecce a una arquitectura RISC, la cual se caracteriza por ejecutar instrucciones simples y rápidas, optimizadas para ofrecer un alto rendimiento con bajo consumo de energía.

Programar en lenguaje ensamblador para el ATmega8535 permite tener control total sobre el hardware, ya que cada instrucción se traduce directamente en operaciones del procesador. Esto resulta fundamental en aplicaciones donde se requiere un uso eficiente de los recursos, un control preciso del tiempo y una comprensión profunda del funcionamiento interno del microcontrolador.

El ensamblador AVR utiliza instrucciones de fácil comprensión y una estructura sencilla, lo que lo convierte en un excelente punto de partida para aprender cómo funcionan internamente los microcontroladores. Algunas de sus características son las siguientes:

- CPU de 8 bits con arquitectura Harvard, que separa la memoria de programa (Flash) de la memoria de datos (SRAM).
- Memoria Flash de 8 KB, SRAM de 512 bytes y EEPROM de 512 bytes.
- 32 registros de propósito general, directamente conectados con la ALU (unidad aritmético-lógica).
- Puertos de entrada/salida (I/O) configurables (PORTA, PORTB, PORTC, PORTD).
- Temporizadores, convertidor ADC de 10 bits, interrupciones, UART, SPI y TWI (I²C) integrados.

La programación en ensamblador para el ATmega8535 se realiza comúnmente usando herramientas como AVR Studio (o AVR Studio 4), donde se pueden escribir, ensamblar y simular los programas antes de cargarlos en el microcontrolador.

Funcionamiento de cada uno de los registros y puerto:

DDRx: Registro de dirección de datos que determina si es entrada o salida.

PORx: Registro para puertos.

Planteamiento del ejercicio:

- a) Desarrollar un programa en ensamblador que lea los valores de los puertos B y D, los compare, y muestre en los puertos A y C los valores según el resultado de la comparación:
- Si los valores son iguales, se muestran ambos en los puertos A y C.
 - Si el valor del puerto B es mayor, se envía a PORTA el valor de B y a PORTC el de D.
 - Si el valor del puerto B es menor, se invierte el orden de salida (D en PORTC y B en PORTA).
- b) El segundo ejercicio consiste en leer un valor numérico desde el Puerto B y convertirlo a su equivalente ASCII hexadecimal, mostrando el resultado en el Puerto A. Esto debe de permitir visualizar un número binario como el carácter correspondiente (0-9 o A-F)

Desarrollo

Material y equipo usado:

- ❖ Jumpers
- ❖ ATmega8535
- ❖ 16 leds
- ❖ 2 dipswitch
- ❖ Fuente de voltaje a 5V
- ❖ 2 pinzas banana caiman

Solución del problema:

Inciso a)

; Programa: Comparación entre los puertos B y D

; Si PINB > PIND -> Muestra PINB en PORTA y PIND en PORTC

; Si PINB < PIND -> Muestra PIND en PORTA y PINB en PORTC

; Si son iguales -> Muestra ambos valores iguales

.include "m8535def.inc" ; Incluye definiciones del microcontrolador ATmega8535

ser r16 ; Carga 0xFF (todos los bits en 1) en R16

out ddra, r16 ; Configura el Puerto A como salida

out ddrc, r16 ; Configura el Puerto C como salida

out portb, r16 ; Activa resistencias pull-up en Puerto B (si es entrada)

out portd, r16 ; Activa resistencias pull-up en Puerto D (si es entrada)

ciclo:

in r16, pinb ; Lee el valor del Puerto B en R16

in r17, pind ; Lee el valor del Puerto D en R17

cp r16, r17 ; Compara los valores de R16 y R17

breq igual ; Si son iguales, salta a la etiqueta 'igual'

brlo menor ; Si R16 < R17, salta a la etiqueta 'menor'

mayor:

out porta, r16 ; Muestra el valor mayor (R16) en PORTA

out portc, r17 ; Muestra el valor menor (R17) en PORTC

rjmp ciclo ; Regresa al inicio del ciclo

igual:

out porta, r16 ; Muestra el valor de R16 en PORTA

```
out portc, r17      ; Muestra el mismo valor en PORTC
rjmp ciclo          ; Regresa al inicio del ciclo
```

menor:

```
out porta, r17      ; Muestra el valor mayor (R17) en PORTA
out portc, r16      ; Muestra el valor menor (R16) en PORTC
rjmp ciclo          ; Regresa al inicio del ciclo
```

Inciso b)

; Programa: Conversión de código ASCII a código HEX (para 7 segmentos o similar)

; Lee un valor de PINB (0-9), lo convierte a su código ASCII

; Si el valor es mayor a 9, lo ajusta para mostrar letras (A-F)

; y muestra el resultado en PORTA.

```
.include "m8535def.inc"      ; Define los registros y puertos del ATmega8535
```

```
.def HEX = R16                ; Define R16 como registro para el valor convertido (salida)
```

```
.def ASC = R17                ; Define R17 como registro para el valor leído (entrada)
```

```
ser R16                       ; R16 = 0xFF
```

```
out DDRA, R16                 ; Configura el Puerto A como salida
```

```
out PORTB, R16                ; Activa resistencias pull-up en Puerto B (entrada)
```

ciclo:

```
in ASC, PINB                  ; Lee el valor del Puerto B (entrada)
```

```
ldi HEX, $30                  ; Carga en HEX el valor base del código ASCII '0' (0x30)
```

```
add HEX, ASC                  ; Suma el valor leído para obtener el código ASCII
correspondiente
```

```
cpi HEX, $3A                  ; Compara si el valor supera el ASCII de '9' (0x39)
```

```
brsh letra                    ; Si es mayor o igual, salta a 'letra'
```

; Si el valor está entre '0' y '9'

guar:

```
out PORTA, HEX                ; Muestra el código ASCII del número en PORTA
```

```
rjmp ciclo                    ; Regresa al inicio del ciclo
```

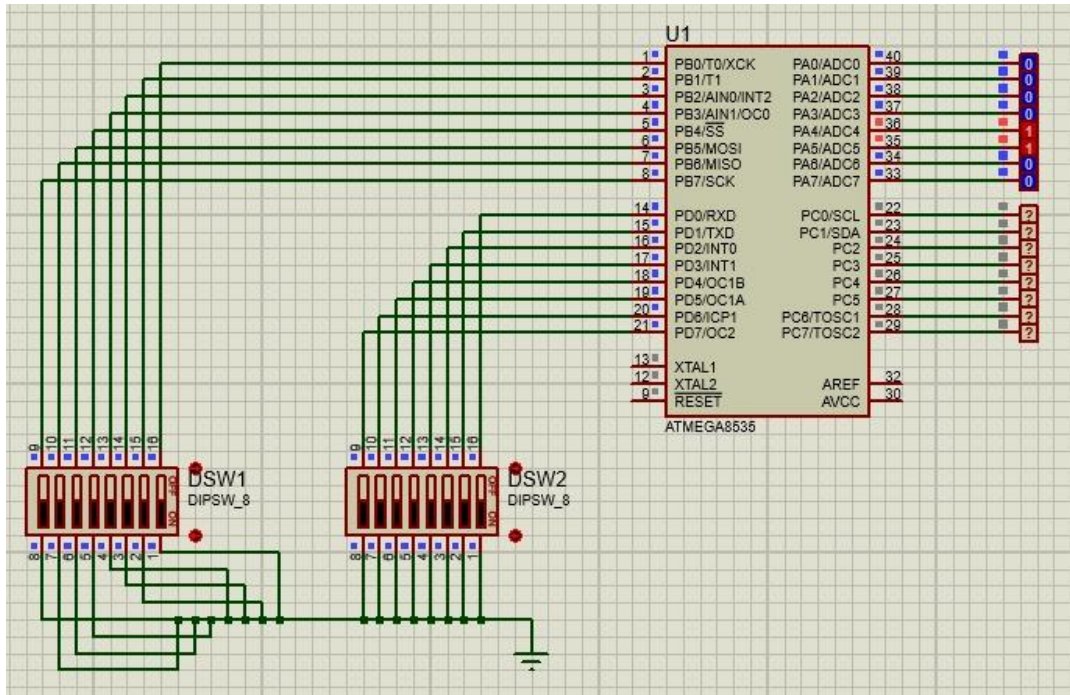
; Si el valor es mayor que 9 (A-F)

letra:

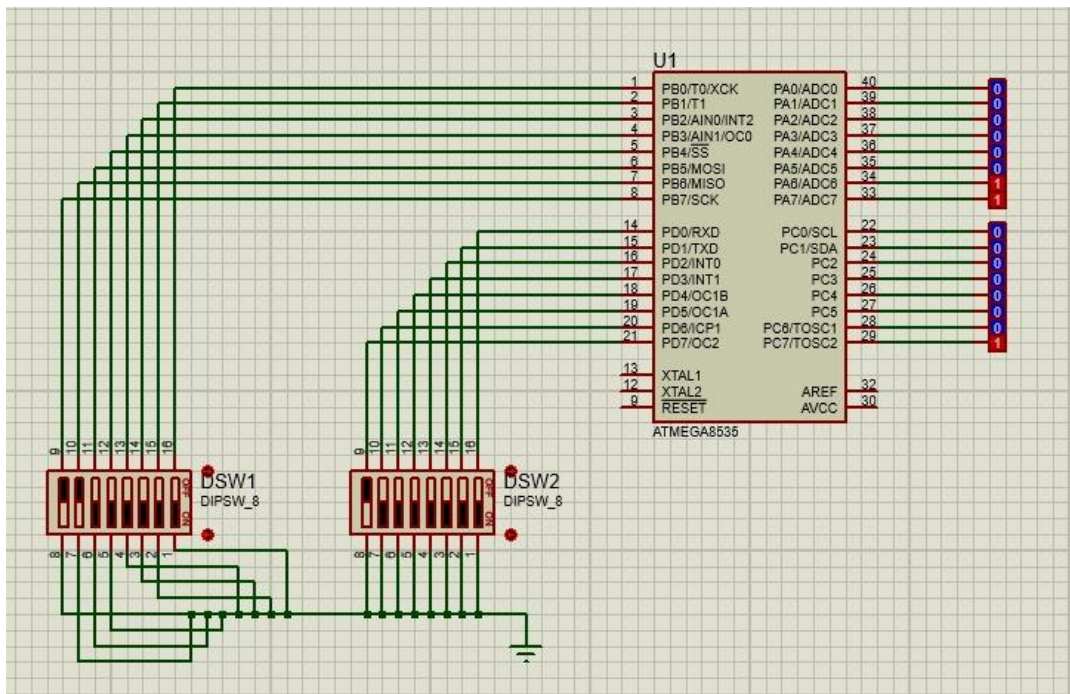
ldi ASC, 7	; Ajuste de 7 para saltar de ':' a 'A' en ASCII
add HEX, ASC	; Corrige el valor para representar letras A-F
rjmp guar	; Va a guardar el resultado en PORTA y repite

Diagrama del circuito:

Inciso a)



Inciso b)



Conclusiones:

Ayala Fuentes Sunem Gizeht:

En esta práctica aplicamos los conocimientos vistos en clase, que con ayuda del profesor logramos llevar a cabo, complicado porque al menos de mi parte fue la primera vez utilizando lenguaje ensamblador, y tomando en cuenta que no manejamos solo un circuito si no dos, creo que lo difícil fue la parte del convertidor ASCII a HEXADECIMAL, dado a que como comente en un inicio el 1er código fue proporcionado por el profesor, pero fue un poco complicado el desarrollar esta práctica.

Además, mediante el comparador, vimos cómo las instrucciones de salto condicional dependen del registro de estado para la toma de decisiones dentro del procesador. Asimismo, en el conversor a ASCII se aplicamos operaciones aritméticas que permitieron transformar valores binarios a su representación hexadecimal, reforzando la comprensión del manejo y presentación de datos.

Guevara Badillo Areli Alejandra:

Esta práctica validó exitosamente la aplicación de conceptos teóricos clave en la programación de microcontroladores.

En el comparador, se demostró cómo la Unidad Aritmético-Lógica (ALU) modifica el registro de estado para que las instrucciones de salto condicional alteren el flujo del programa. Este es el principio fundamental de la toma de decisiones en un procesador.

En el conversor a ASCII, se aplicó la manipulación de datos. Usando operaciones aritméticas, se sumó un valor de desplazamiento (offset) para transformar un número binario a su estándar de representación textual, un proceso esencial para la interfaz con otros sistemas.

Leon Perez May Carolina:

La práctica permitió comprender de mejor manera el flujo de un programa hecho en ensamblador. Este ayudo a poder aprender de manera practica la utilización de entradas y salidas con el ATMEGA8535, lo cual es fundamental para manejar en prácticas más avanzadas.

Por otra parte, aprender a trabajar con valores hexadecimales de manera rápida fue un reto al igual que manejar los offset en este mismo formato.

Referencias:

- ❖ *ATmega8535*. (s/f). Ehu.es. Recuperado el 31 de octubre de 2025, de [http://www.sc.ehu.es/sbweb/webcentro/automatica/web_avr/archivos/Otros%20AVRs/ATmega/ATmega8535.htm#:~:text=Operaci%C3%B3n%20de%20lectura%20durante%20la,prescaler%20separado%20y%20modo%20comparaci%C3%B3n.&text=modo%20de%20captura.,10%20bit%20y%208%20canales.&text=7%20canales%20diferenciales%20s%C3%B3lo%20en%20el%20encapsulado%20TQFP.&text=s%C3%B3lo%20en%20el-.encapsulado%20TQFP.,down%2C%20Standby%20y%20Standby%20extendido.&text=%2D%2032%20I%C3%ADneas%20de%20I/O,%2D%2016%20MHz%20\(ATmega8535\)](http://www.sc.ehu.es/sbweb/webcentro/automatica/web_avr/archivos/Otros%20AVRs/ATmega/ATmega8535.htm#:~:text=Operaci%C3%B3n%20de%20lectura%20durante%20la,prescaler%20separado%20y%20modo%20comparaci%C3%B3n.&text=modo%20de%20captura.,10%20bit%20y%208%20canales.&text=7%20canales%20diferenciales%20s%C3%B3lo%20en%20el%20encapsulado%20TQFP.&text=s%C3%B3lo%20en%20el-.encapsulado%20TQFP.,down%2C%20Standby%20y%20Standby%20extendido.&text=%2D%2032%20I%C3%ADneas%20de%20I/O,%2D%2016%20MHz%20(ATmega8535))
- ❖ *ATMEGA8535 T unida 1*. (s/f). Scribd. Recuperado el 31 de octubre de 2025, de <https://es.scribd.com/doc/51490004/ATMEGA8535tUnida1>
- ❖ *ATmega8535*. (s/f-b). Microchip.com. Recuperado el 31 de octubre de 2025, de <https://www.microchip.com/en-us/product/atmega8535>
- ❖ *ATmega microcontrollers (MCUs)*. (s/f). Microchip.com. Recuperado el 31 de octubre de 2025, de <https://www.microchip.com/en-us/about/corporate-overview/acquisitions/atmel/atmega>
- ❖ *Microcontrolador ATMEGA8535-16PU*. (s/f). Componentes Electronicos Aragon. Recuperado el 31 de octubre de 2025, de <https://aragonelectronica.com/products/microcontrolador-atmega8535-16pu>
- ❖ Temas vistos en clase